

SQL interview guide

For campus placement rounds: quick syntax reference, joins, subqueries and CTEs, window functions, NULLs, indexes, normalisation, transactions and 15 classic interview problems with solutions.

How to use this guide

- Every query in this guide uses the **same four tables** (below). Create them once in MySQL 8+ or PostgreSQL and run each query yourself.
- Code is standard SQL. Where MySQL and PostgreSQL behave differently, it is marked **MySQL** / **PostgreSQL**.
- In interviews, say your approach out loud before writing: "I will group by department, then filter groups with HAVING." Interviewers care about reasoning as much as syntax.

Sample schema (used everywhere)

```
CREATE TABLE departments (  
  dept_id    INT PRIMARY KEY,  
  dept_name  VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE employees (  
  emp_id     INT PRIMARY KEY,  
  name       VARCHAR(50) NOT NULL,  
  dept_id    INT REFERENCES departments(dept_id),  
  manager_id INT REFERENCES employees(emp_id),  
  salary     INT NOT NULL,  
  hire_date  DATE NOT NULL  
);  
  
CREATE TABLE orders (  
  order_id   INT PRIMARY KEY,  
  emp_id     INT REFERENCES employees(emp_id),  
  order_date DATE NOT NULL,  
  amount     INT NOT NULL  
);  
  
CREATE TABLE logins (  
  log_id     INT PRIMARY KEY,  
  user_id    INT NOT NULL,  
  login_date DATE NOT NULL  
);  
  
INSERT INTO departments VALUES  
  (10, 'Engineering'), (20, 'Sales'), (30, 'HR'), (40, 'Marketing');  
  
INSERT INTO employees VALUES  
  (1, 'Asha', 10, NULL, 120000, '2019-04-01'),  
  (2, 'Ravi', 10, 1, 90000, '2020-07-15'),  
  (3, 'Meena', 10, 1, 90000, '2021-01-10'),  
  (4, 'Karan', 20, 1, 70000, '2020-02-20'),  
  (5, 'Divya', 20, 4, 75000, '2022-06-01'),  
  (6, 'Imran', 30, 1, 50000, '2023-03-05'),  
  (7, 'Neha', NULL, 4, 40000, '2024-08-12');  
  
INSERT INTO orders VALUES  
  (101, 4, '2026-01-01', 500),  
  (102, 5, '2026-01-02', 300),  
  (103, 4, '2026-01-02', 200),  
  (104, 5, '2026-01-05', 700);  
  
INSERT INTO logins VALUES  
  (1, 1, '2026-03-01'), (2, 1, '2026-03-02'), (3, 1, '2026-03-02'),  
  (4, 1, '2026-03-03'), (5, 1, '2026-03-05'),  
  (6, 2, '2026-03-01'), (7, 2, '2026-03-03');
```

Things built into this data on purpose: Marketing has **no employees**, Neha has **no department** (NULL), Ravi and Meena have the **same salary** (a tie), Divya earns **more than her manager** Karan, and logins has one

duplicate row (log_id 3).

1. Core syntax quick reference

Written order vs execution order

You write a query in one order, but the database logically runs it in another. This explains most "why doesn't my alias work" errors.

Step	Clause	What happens
1	FROM / JOIN	Pick tables and combine rows
2	WHERE	Filter individual rows
3	GROUP BY	Make groups
4	HAVING	Filter whole groups
5	SELECT	Compute columns, aliases and window functions
6	DISTINCT	Remove duplicate result rows
7	ORDER BY	Sort
8	LIMIT / OFFSET	Keep only some rows

- A column alias from SELECT **cannot** be used in WHERE (WHERE runs first). It **can** be used in ORDER BY.
- MySQL also accepts SELECT aliases in GROUP BY and HAVING; PostgreSQL accepts them in GROUP BY but not HAVING. To be safe, repeat the expression.
- Window functions are computed in step 5, so you cannot filter on them in WHERE. Wrap the query in a subquery or CTE first.

WHERE vs HAVING

	WHERE	HAVING
Filters	Rows, before grouping	Groups, after grouping
Aggregates allowed?	No (WHERE COUNT(*) > 1 is an error)	Yes
Example	WHERE salary > 60000	HAVING COUNT(*) > 1

```
-- Departments with more than one employee earning over 60000
SELECT dept_id, COUNT(*) AS cnt
FROM employees
WHERE salary > 60000          -- row filter first
GROUP BY dept_id
HAVING COUNT(*) > 1;          -- group filter after
-- Result: dept 10 (3), dept 20 (2)
```

GROUP BY rules

- Every column in SELECT must be either inside an aggregate (COUNT , SUM , AVG , MIN , MAX) or listed in GROUP BY.
- PostgreSQL enforces this strictly. MySQL enforces it when ONLY_FULL_GROUP_BY is on (the default since 5.7). Older MySQL silently picked a random value, which is a classic bug.
- All NULLs fall into one group. In our data, Neha forms her own dept_id = NULL group.

ORDER BY and LIMIT

- ORDER BY salary DESC, name ASC sorts by salary, then breaks ties by name.
- Without ORDER BY, row order is **not guaranteed**, even if it looks stable.
- LIMIT 5 OFFSET 10 skips 10 rows and returns the next 5 (MySQL and PostgreSQL). MySQL also allows LIMIT 10, 5 (offset first).
- Standard SQL form: FETCH FIRST 5 ROWS ONLY (PostgreSQL, Oracle 12c+). SQL Server uses TOP 5 .

Small but commonly asked

- **UNION vs UNION ALL**: UNION removes duplicates (extra sort/hash work); UNION ALL keeps all rows and is faster.
- **DELETE vs TRUNCATE vs DROP**: DELETE removes chosen rows (can use WHERE, fires triggers); TRUNCATE removes all rows quickly; DROP removes the table itself. In MySQL, TRUNCATE commits implicitly and cannot be rolled back; in PostgreSQL it can be rolled back inside a transaction.
- **Integer division**: in PostgreSQL 5 / 2 is 2 ; in MySQL it is 2.5000 (use DIV for integer division). Write 100.0 * a / b to force decimals in both.
- **Primary key vs unique key**: a table has one primary key (no NULLs); it can have many unique keys, and unique columns may hold NULLs.

2. Joins

To keep results small, take only three employees and three departments:

employees (subset)		departments (subset)	
name	dept_id	dept_id	dept_name
Asha	10	10	Engineering
Karan	20	20	Sales
Neha	NULL	40	Marketing

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON d.dept_id = e.dept_id;
```

Join type	Rows returned (name, dept_name)	Count
INNER JOIN	(Asha, Engineering), (Karan, Sales)	2
LEFT JOIN	Inner rows + (Neha, NULL)	3
RIGHT JOIN	Inner rows + (NULL, Marketing)	3
FULL OUTER JOIN	Inner rows + (Neha, NULL) + (NULL, Marketing)	4
CROSS JOIN	Every employee with every department	3 x 3 = 9

- **INNER:** only matching rows. Neha's NULL never equals anything, so she is dropped.
- **LEFT:** all rows from the left table; NULLs where there is no match. **RIGHT** is the mirror image (rarely used; just swap the tables and use LEFT).
- **FULL OUTER:** all rows from both sides. **MySQL does not support it**; emulate it:

```
SELECT e.name, d.dept_name
FROM employees e LEFT JOIN departments d ON d.dept_id = e.dept_id
UNION ALL
SELECT e.name, d.dept_name
FROM employees e RIGHT JOIN departments d ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;  -- only the right-side rows with no match
```

- **SELF JOIN:** a table joined to itself with two aliases, e.g. employee and manager:

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.emp_id = e.manager_id;
-- Asha gets manager NULL (she has none); use INNER JOIN to drop her.
```

Tip: In a LEFT JOIN, a filter on the right table belongs in `ON`, not `WHERE`. `LEFT JOIN orders o ON o.emp_id = e.emp_id WHERE o.amount > 400` removes employees with no orders (their `o.amount` is NULL), so it behaves like an INNER JOIN. Writing `ON o.emp_id = e.emp_id AND o.amount > 400` keeps every employee.

3. Subqueries vs CTEs

Kinds of subqueries

Kind	Example
Scalar (one value)	<code>WHERE salary > (SELECT AVG(salary) FROM employees)</code>
Multi-row	<code>WHERE dept_id IN (SELECT dept_id FROM departments WHERE dept_name LIKE 'S%')</code>
Correlated (runs per outer row)	<code>WHERE salary > (SELECT AVG(salary) FROM employees x WHERE x.dept_id = e.dept_id)</code>
EXISTS	<code>WHERE EXISTS (SELECT 1 FROM orders o WHERE o.emp_id = e.emp_id)</code>
Derived table (in FROM)	<code>FROM (SELECT ...) AS t</code> (always give it an alias)

CTE (Common Table Expression)

A CTE is a named subquery written at the top with `WITH`. It makes long queries readable and can be referenced more than once. Supported in PostgreSQL and MySQL 8.0+.

```
WITH dept_avg AS (  
  SELECT dept_id, AVG(salary) AS avg_sal  
  FROM employees  
  GROUP BY dept_id  
)  
SELECT e.name, e.salary, a.avg_sal  
FROM employees e  
JOIN dept_avg a ON a.dept_id = e.dept_id  
WHERE e.salary > a.avg_sal;  
-- Asha (120000 > 100000), Divya (75000 > 72500)
```

Recursive CTE (hierarchy)

```
WITH RECURSIVE chain AS (  
  SELECT emp_id, name, 1 AS lvl  
  FROM employees  
  WHERE manager_id IS NULL -- anchor: the top boss  
  UNION ALL  
  SELECT e.emp_id, e.name, c.lvl + 1  
  FROM employees e  
  JOIN chain c ON e.manager_id = c.emp_id -- step: direct reports  
)  
SELECT name, lvl FROM chain ORDER BY lvl, name;  
-- Asha 1 | Imran, Karan, Meena, Ravi 2 | Divya, Neha 3
```

Subquery	CTE
Written inline, can nest deeply	Named once at the top, reads top to bottom
Must be repeated to reuse	Can be referenced many times in the query
Cannot be recursive	Can be recursive (<code>WITH RECURSIVE</code>)

Performance is usually similar; modern optimisers often inline CTEs. Choose the one that is easier to read.

4. Window functions

A window function computes a value **across related rows without collapsing them** (unlike GROUP BY).

Syntax: `FUNC() OVER (PARTITION BY ... ORDER BY ... [frame])`. Supported in PostgreSQL and MySQL 8.0+.

ROW_NUMBER vs RANK vs DENSE_RANK

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
       RANK()      OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employees;
```

name	salary	ROW_NUMBER	RANK	DENSE_RANK
Asha	120000	1	1	1
Ravi	90000	2	2	2
Meena	90000	3	2	2
Divya	75000	4	4	3
Karan	70000	5	5	4
Imran	50000	6	6	5
Neha	40000	7	7	6

- **ROW_NUMBER**: always unique 1, 2, 3... Ties get different numbers in an arbitrary order (Ravi and Meena may swap). Add a tie-breaker like `ORDER BY salary DESC, emp_id`.
- **RANK**: ties share a rank, then a gap (2, 2, 4).
- **DENSE_RANK**: ties share a rank, no gap (2, 2, 3). Use it for "Nth highest salary".
- Add `PARTITION BY dept_id` to restart numbering for each department.

LAG and LEAD

`LAG(col, n)` reads the value n rows before the current row; `LEAD(col, n)` reads n rows after. Default n is 1; outside the partition you get NULL (or a default given as the third argument).

```
SELECT emp_id, order_date, amount,
       LAG(amount) OVER (PARTITION BY emp_id ORDER BY order_date) AS prev_amt,
       LEAD(amount) OVER (PARTITION BY emp_id ORDER BY order_date) AS next_amt
FROM orders;
```

emp_id	order_date	amount	prev_amt	next_amt
4	2026-01-01	500	NULL	200
4	2026-01-02	200	500	NULL
5	2026-01-02	300	NULL	700
5	2026-01-05	700	300	NULL

Running totals

```
SELECT order_id, order_date, amount,
       SUM(amount) OVER (ORDER BY order_date, order_id
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM orders;
-- 101: 500 | 102: 800 | 103: 1000 | 104: 1700
```

Tip: If you write only `OVER (ORDER BY order_date)`, the default frame is `RANGE`, which treats rows with the same date as one step. Orders 102 and 103 (both on 2026-01-02) would *both* show 1000. Use a unique `ORDER BY` and an explicit `ROWS` frame for a true row-by-row running total.

Other useful ones

- `SUM(salary) OVER (PARTITION BY dept_id)`: department total on every row (no `ORDER BY`, so the whole partition).
- `AVG(amount) OVER (ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`: 3-row moving average.
- `NTILE(4) OVER (ORDER BY salary)`: splits rows into 4 roughly equal buckets (quartiles).
- `FIRST_VALUE(name) OVER (PARTITION BY dept_id ORDER BY salary DESC)`: top earner's name on every row of the department.

5. NULL handling

NULL means "unknown", not zero or empty string. Any comparison with NULL gives **UNKNOWN**, and `WHERE` keeps only TRUE rows.

Expression	Result	Correct way
<code>NULL = NULL</code>	UNKNOWN (row not returned)	<code>col IS NULL</code>
<code>dept_id <> 10</code>	Skips Neha (NULL)	<code>dept_id <> 10 OR dept_id IS NULL</code>
<code>salary + NULL</code>	NULL	<code>salary + COALESCE(bonus, 0)</code>
<code>x / 0</code>	Error in PostgreSQL, NULL in MySQL	<code>x / NULLIF(y, 0)</code> gives NULL safely

- `COUNT(*)` counts all rows; `COUNT(dept_id)` skips NULLs. On our table: 7 vs 6.
- `SUM`, `AVG`, `MIN`, `MAX` ignore NULLs. So `AVG(bonus)` divides by the non-NULL count, not by all rows.
- `COALESCE(a, b, c)` returns the first non-NULL value. `NULLIF(a, b)` returns NULL if `a = b`, else `a`.
- Sorting: NULLs come **first** in ascending order in MySQL and **last** in PostgreSQL. PostgreSQL supports `NULLS FIRST` / `NULLS LAST`; MySQL does not.
- String joining: `CONCAT('a', NULL)` is NULL in MySQL but `'a'` in PostgreSQL; `'a' || NULL` is NULL in PostgreSQL.
- **The NOT IN trap:** if the subquery returns even one NULL, `NOT IN` returns no rows. See problem 6. Prefer `NOT EXISTS`.

6. Indexes and normalisation

Indexes

An index is a separate sorted structure (usually a B-tree) that lets the database find rows without scanning the whole table, like a book index.

- **Trade-off:** faster reads (WHERE, JOIN, ORDER BY), but slower INSERT/UPDATE/DELETE and extra storage. Do not index every column.
- **Good candidates:** columns used in WHERE and JOIN often, with many distinct values (e.g. `email`, foreign keys). A column like `gender` rarely benefits.
- **Composite index** (`dept_id, salary`) helps `WHERE dept_id = 10` and `WHERE dept_id = 10 AND salary > 50000`, but generally not `WHERE salary > 50000` alone (leftmost-prefix rule).
- **Index killers:** wrapping the column in a function (`WHERE YEAR(hire_date) = 2020`; write `hire_date >= '2020-01-01' AND hire_date < '2021-01-01'` instead) and leading wildcards (`LIKE '%sha'`; `LIKE 'As%'` can use the index).
- **Clustered vs non-clustered:** a clustered index stores the table rows themselves in key order (only one per table). MySQL InnoDB clusters by the primary key. Other indexes are non-clustered and point to the rows. PostgreSQL tables are heaps, and every index is a separate structure.
- Use `EXPLAIN` (or `EXPLAIN ANALYZE`) to see whether a query uses an index.

Normal forms

Form	Rule	Violation example and fix
1NF	Atomic values; no repeating groups or lists in a cell	<code>phones = '98...', '99...'</code> in one cell. Fix: one row per phone in a separate table.
2NF	1NF + no non-key column depends on <i>part</i> of a composite key	<code>order_items(order_id, product_id, product_name, qty)</code> : <code>product_name</code> depends only on <code>product_id</code> . Fix: move it to <code>products</code> .
3NF	2NF + no non-key column depends on another non-key column (no transitive dependency)	<code>employees(emp_id, dept_id, dept_name)</code> : <code>dept_name</code> depends on <code>dept_id</code> . Fix: separate <code>departments</code> table (as in our schema).
BCNF	For every dependency $X \rightarrow Y$, X must be a candidate key	<code>(student, subject, teacher)</code> , key <code>(student, subject)</code> , each teacher teaches one subject: <code>teacher → subject</code> , but <code>teacher</code> is not a key. Fix: split into <code>(teacher, subject)</code> and <code>(student, teacher)</code> .

Denormalisation (adding some redundancy on purpose) is sometimes used for read-heavy reporting to avoid expensive joins. Mention this trade-off if asked "is normalisation always good?"

7. Transactions and ACID

A transaction is a group of statements that succeed or fail as one unit.

```
BEGIN;  -- MySQL also accepts START TRANSACTION
UPDATE accounts SET balance = balance - 1000 WHERE acc_id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE acc_id = 2;
COMMIT;  -- or ROLLBACK; to undo both
```

Property	Meaning	In the transfer example
Atomicity	All or nothing	Never debit without credit
Consistency	Moves the database from one valid state to another; constraints hold	A <code>CHECK (balance >= 0)</code> is never broken
Isolation	Concurrent transactions do not see each other's half-done work	Another user never sees money "missing" mid-transfer
Durability	Once committed, data survives a crash	Written to the log on disk before COMMIT returns

Isolation levels and anomalies

Level	Dirty read	Non-repeatable read	Phantom read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible (per the standard)
Serializable	Prevented	Prevented	Prevented

- **Dirty read**: reading another transaction's uncommitted change. **Non-repeatable read**: the same row gives a different value when read again. **Phantom**: re-running a query returns new rows.
- Defaults: PostgreSQL uses **Read Committed**; MySQL InnoDB uses **Repeatable Read**. Both actually prevent most phantoms at Repeatable Read, stricter than the standard requires.
- A **deadlock** happens when two transactions each wait for a lock the other holds. The database detects it and rolls one back; the application should retry.

8. Classic interview problems (with solutions)

All use the sample schema. Try each one yourself before reading the answer.

Q1. Find the second highest salary. Return NULL if it does not exist.

```
-- Method 1: works everywhere, returns NULL if none
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Method 2: LIMIT/OFFSET, wrapped so an empty result becomes NULL
SELECT (SELECT DISTINCT salary FROM employees
        ORDER BY salary DESC LIMIT 1 OFFSET 1) AS second_highest;
-- Result: 90000
```

Note the `DISTINCT`: without it, a tie at the top would make the "second" value equal the first.

Q2. Find the Nth highest salary (say N = 3).

```
SELECT DISTINCT salary
FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) AS t
WHERE rnk = 3;
-- Result: 75000 (120000 is 1st, 90000 is 2nd)
```

Use `DENSE_RANK`, not `ROW_NUMBER` or `RANK`, so ties count as one salary level.

Q3. Top 2 earners in each department.

```
WITH ranked AS (  
    SELECT d.dept_name, e.name, e.salary,  
           DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS rnk  
    FROM employees e  
    JOIN departments d ON d.dept_id = e.dept_id  
)  
SELECT dept_name, name, salary  
FROM ranked  
WHERE rnk <= 2  
ORDER BY dept_name, salary DESC;  
-- Engineering: Asha 120000, Ravi 90000, Meena 90000  
-- HR: Imran 50000 | Sales: Divya 75000, Karan 70000
```

Ask the interviewer how ties should be handled: DENSE_RANK keeps both Ravi and Meena; ROW_NUMBER returns exactly 2 rows per department.

Q4. Employees who earn more than their manager.

```
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_salary  
FROM employees e  
JOIN employees m ON m.emp_id = e.manager_id  
WHERE e.salary > m.salary;  
-- Divya 75000, manager Karan 70000
```

Q5. Employees earning more than the average of their own department.

```
SELECT e.name, e.dept_id, e.salary  
FROM employees e  
WHERE e.salary > (  
    SELECT AVG(x.salary) FROM employees x WHERE x.dept_id = e.dept_id  
)  
-- Asha (dept avg 100000), Divya (dept avg 72500)
```

This is a correlated subquery. The CTE version in section 3 gives the same result and is often faster on large tables.

Q6. Departments with no employees.

```
-- Correct: NOT EXISTS
SELECT d.dept_name
FROM departments d
WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);

-- Also correct: LEFT JOIN + IS NULL (anti-join)
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
-- Result: Marketing

-- WRONG: returns zero rows, because Neha's dept_id is NULL
SELECT dept_name FROM departments
WHERE dept_id NOT IN (SELECT dept_id FROM employees);
```

40 NOT IN (10, 20, 30, NULL) means 40 <> 10 AND ... AND 40 <> NULL, and the last part is UNKNOWN, so no row passes.

Q7. Department-wise headcount and average salary, only for departments with more than one employee.

```
SELECT d.dept_name, COUNT(*) AS headcount, ROUND(AVG(e.salary), 2) AS avg_salary
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING COUNT(*) > 1;
-- Engineering 3 100000.00 | Sales 2 72500.00
```

Q8. Managers with at least 3 direct reports.

```
SELECT m.name, COUNT(*) AS reports
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
GROUP BY m.emp_id, m.name
HAVING COUNT(*) >= 3;
-- Asha 4 (Ravi, Meena, Karan, Imran). Karan has 2, so he is excluded.
```

Q9. Employees who have never placed an order.

```
SELECT e.name
FROM employees e
LEFT JOIN orders o ON o.emp_id = e.emp_id
WHERE o.order_id IS NULL;
-- Asha, Ravi, Meena, Imran, Neha
```

Q10. Find duplicate rows (same user, same login date).

```
SELECT user_id, login_date, COUNT(*) AS times
FROM logins
GROUP BY user_id, login_date
HAVING COUNT(*) > 1;
-- user 1, 2026-03-02, 2
```

Q11. Delete duplicates, keeping the row with the smallest log_id.

```
-- PostgreSQL
DELETE FROM logins a
USING logins b
WHERE a.user_id = b.user_id
      AND a.login_date = b.login_date
      AND a.log_id > b.log_id;

-- MySQL
DELETE a FROM logins a
JOIN logins b
  ON a.user_id = b.user_id
  AND a.login_date = b.login_date
  AND a.log_id > b.log_id;
-- Deletes log_id 3
```

The common form `DELETE ... WHERE log_id NOT IN (SELECT MIN(log_id) FROM logins GROUP BY ...)` works in PostgreSQL but fails in MySQL (error 1093: you cannot select from the table you are deleting from) unless you wrap the subquery in another derived table.

Q12. Users who logged in on 3 or more consecutive days.

Idea ("gaps and islands"): number each user's distinct dates 1, 2, 3... Subtract that number of days from the date. Consecutive dates give the **same** result, so they form one group.

```
WITH d AS (  
    SELECT DISTINCT user_id, login_date FROM logins  
)  
g AS (  
    SELECT user_id, login_date,  
           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn  
    FROM d  
)  
SELECT user_id, MIN(login_date) AS start_date, MAX(login_date) AS end_date,  
       COUNT(*) AS days  
FROM g  
GROUP BY user_id, login_date - CAST(rn AS INT)      -- PostgreSQL  
-- MySQL: GROUP BY user_id, DATE_SUB(login_date, INTERVAL rn DAY)  
HAVING COUNT(*) >= 3;  
-- user 1, 2026-03-01 to 2026-03-03, 3 days
```

user 1 date	rn	date - rn days
03-01	1	02-28
03-02	2	02-28
03-03	3	02-28
03-05	4	03-01

The DISTINCT step matters: without it, the duplicate 03-02 row would break the pattern.

Q13. Daily sales and change from the previous sales day.

```
WITH daily AS (  
    SELECT order_date, SUM(amount) AS total  
    FROM orders  
    GROUP BY order_date  
)  
SELECT order_date, total,  
       total - LAG(total) OVER (ORDER BY order_date) AS change_from_prev  
FROM daily  
ORDER BY order_date;  
-- 2026-01-01 500 NULL | 2026-01-02 500 0 | 2026-01-05 700 200
```

Q14. Per department: total headcount and how many earn 80000 or more (pivot-style).

```
SELECT d.dept_name,  
       COUNT(e.emp_id) AS headcount,  
       SUM(CASE WHEN e.salary >= 80000 THEN 1 ELSE 0 END) AS high_paid  
FROM departments d  
LEFT JOIN employees e ON e.dept_id = d.dept_id  
GROUP BY d.dept_name  
ORDER BY d.dept_name;  
-- Engineering 3 3 | HR 1 0 | Marketing 0 0 | Sales 2 0
```

`COUNT(e.emp_id)` (not `COUNT(*)`) gives 0 for Marketing, because its joined row has a NULL `emp_id`. This "SUM of CASE" trick is how you turn rows into columns in any database.

Q15. Each employee's salary as a percentage of their department's total.

```
SELECT name, dept_id, salary,  
       ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY dept_id), 2) AS pct_of_dept  
FROM employees  
WHERE dept_id IS NOT NULL  
ORDER BY dept_id, salary DESC;  
-- Asha 40.00, Ravi 30.00, Meena 30.00, Divya 51.72, Karan 48.28, Imran 100.00
```

The `100.0` avoids integer division in PostgreSQL. A window SUM keeps every row, which GROUP BY cannot do.

9. Quick theory questions

What is the difference between a view and a table?

A view is a saved SELECT query; it stores no data of its own (except a materialized view, which stores the result and must be refreshed). Views simplify complex queries and can hide columns for security.

Primary key vs foreign key?

A primary key uniquely identifies each row in its own table (unique, not NULL). A foreign key in one table refers to the primary (or unique) key of another table and enforces referential integrity, e.g.

`employees.dept_id` must exist in `departments`.

CHAR vs VARCHAR?

`CHAR(n)` is fixed length (padded with spaces); good for fixed codes like country codes. `VARCHAR(n)` stores only the actual length, up to n.

DDL, DML, DCL, TCL?

DDL defines structure (CREATE, ALTER, DROP, TRUNCATE). DML changes data (INSERT, UPDATE, DELETE; SELECT is often grouped here or called DQL). DCL controls access (GRANT, REVOKE). TCL controls transactions (COMMIT, ROLLBACK, SAVEPOINT).

IN vs EXISTS?

IN compares a value against a list; EXISTS checks whether a subquery returns any row and stops at the first match. Results are the same for positive checks, and modern optimisers often plan them alike. For the negative form, prefer NOT EXISTS because NOT IN breaks with NULLs.

Last-week checklist

- Can you write the execution order from memory and explain why an alias fails in WHERE?
- Can you draw the INNER / LEFT / FULL results for two 3-row tables in under a minute?
- Solve Q1-Q4 and Q6 without looking: second highest, Nth highest, top N per group, more than manager, no employees.
- Know ROW_NUMBER vs RANK vs DENSE_RANK with a tie example, and when to use each.
- Explain the NOT IN with NULL trap and the ON vs WHERE trap in LEFT JOIN.
- Write one running total and one LAG query with PARTITION BY.
- Say 1NF, 2NF, 3NF and BCNF in one line each, with one example.
- Explain ACID with the bank transfer, and name the two default isolation levels (PostgreSQL: Read Committed, MySQL: Repeatable Read).
- Know which database the company's test uses; note the MySQL vs PostgreSQL differences (FULL JOIN, LIMIT forms, NULL sort order, integer division).
- Practise typing queries without autocomplete; many online tests use a plain editor.

PrepSuccess prep guide · Practise these on PrepSuccess: Skill checks, Learn tasks and Interview prep.